



CSCE 633-700: Machine Learning

Linear regression

Overview

Linear Regression

- Example
- Representation, Evaluation
- Optimization: Closed form solution via Ordinary Least Squares
- Optimization: Numerical solution via Gradient Descent
 - General gradient descent
 - Gradient descent for linear regression (batch, stochastic, mini-batch)
- Non-linear basis function for regression & Overfitting

[Parts of these slides have been adapted from K. Murphy (Machine Learning: A probabilistic perspective), Dr. Andrew Ng's Machine Learning course at Coursera, and CSCI567 Machine Learning (USC, Drs. Sha & Liu)]

Linear Regression: Computational Complexity

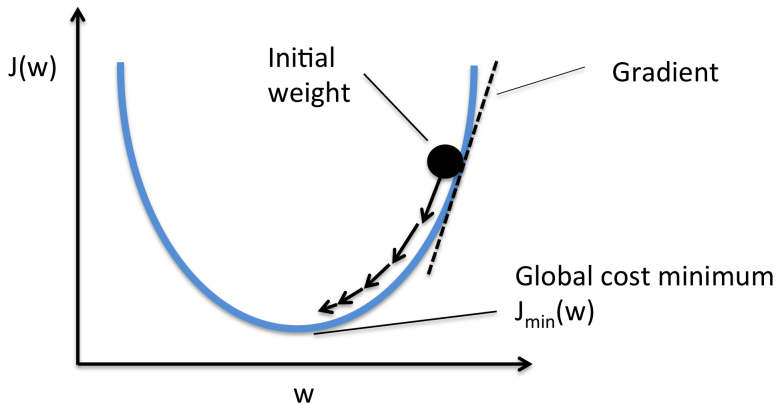
- Bottleneck for computing the solution $\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ is to invert the matrix $\mathbf{X}^T \mathbf{X} \in \mathbb{R}^{D \times D}$
- Computational complexity is $O((D + 1)^3)$ using Gauss-Jordan elimination
 - Impractical for large D
- Alternative
 - Find **approximate** solution through an iterative optimization algorithm
 - e.g. **Gradient Descent**

Gradient Descent

- Iterative algorithm finding a function's minimum via local search
- Standard optimization algorithm in many ML applications
 - e.g. linear regression, logistic regression
 - scales well to large datasets (e.g. no matrix multiplication)
 - proof that it solves many convex problems
 - good heuristic to non-convex problems as well
- **Input:** Function $J(\mathbf{w}) \in \mathbb{R}$
- **Output:** Local minimum $\mathbf{w}^*$
- **Goal:** Minimize $J(\mathbf{w})$ via greedy local search

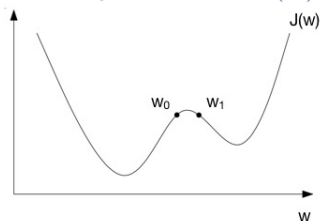
Gradient Descent

1-dimensional example: $J(w) = w^2$



Gradient Descent: 1-dimensional case

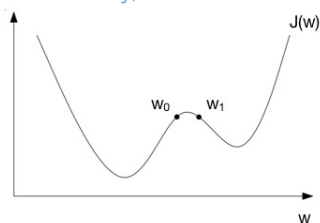
What will happen if we try to minimize $J(w)$ via a local search?



- Starting from w_0
 - We look to the right ($J(w) \uparrow$) and to the left ($J(w) \downarrow$)
 - We take a small step to the left
 - We repeat the above until we reach the **left basin**
- Starting from w_1
 - We similarly reach the **right basin**
- It is clear that the outcome depends on the **starting point**

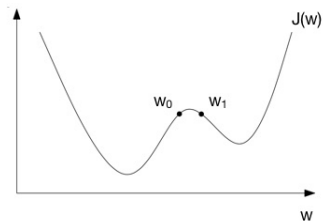
Gradient Descent: 1-dimensional case

More formally, we do the following



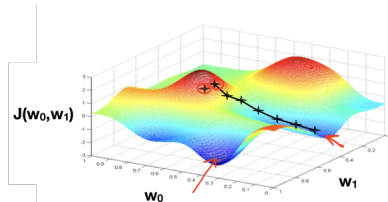
- $J'(w) = \frac{dj(w)}{dw} \approx \frac{J(w+\epsilon) - J(w)}{\epsilon}$, for $\epsilon \rightarrow 0$ (def. 1st order derivative)
- While $J'(w) \neq 0$
 - If $J'(w) > 0$ (i.e. $J(w + \epsilon) > J(w)$ and $J(w) \uparrow$), move w a little bit to the left
 - If $J'(w) < 0$ (i.e. $J(w + \epsilon) < J(w)$ and $J(w) \downarrow$), move w a little bit to the right
- The derivative $J'(w)$ is used to decide which direction to move
- In other words, move w towards the direction of $-J'(w)$

Gradient Descent: Algorithm Outline



1-dimensional

- 1 Initialize w , ϵ , $\alpha(\cdot)$, $k := 0$
- 2 While $|\frac{dJ(w)}{dw}| > \epsilon$
 - 2a $k := k + 1$
 - 2b $w := w - \alpha(k) \cdot \frac{dJ(w)}{dw}$

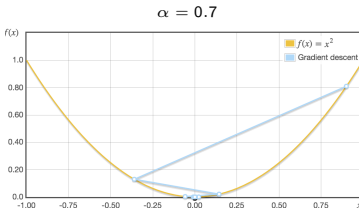
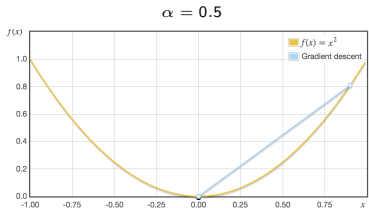
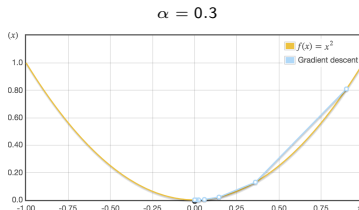
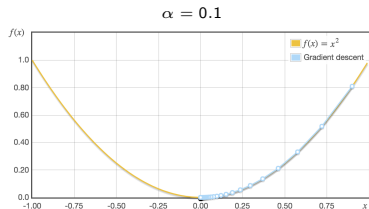


[Source: Machine Learning, Coursera, Andrew Ng]

d-dimensional

- 1 Initialize $\mathbf{w}$, ϵ , $\alpha(\cdot)$, $k := 0$
- 2 While $\|\nabla J(\mathbf{w})\|_2 > \epsilon$
 - 2a $k := k + 1$
 - 2b $\mathbf{w} := \mathbf{w} - \alpha(k) \cdot \nabla J(\mathbf{w})$

Gradient Descent: Step size (or learning rate) α



- If $\alpha(k)$ too small, convergence is unnecessarily slow
- If $\alpha(k)$ too large, correction process will overshoot and can diverge

Gradient Descent: Step size (or learning rate) α

How to chose α ?

- In practice, through experimentation
 - Check how $J(\mathbf{w})$ behaves over iterations for multiple α
 - α is a **hyper-parameter**
 - Therefore it can be tuned using a **dev-set** or a **cross-validation** framework

Gradient Descent: Stopping rule

- Hyper-parameter ϵ (i.e. $\|\nabla J(\mathbf{w})\|_2 > \epsilon$) determines when to stop
- Small ϵ : many iterations but higher quality solution
- Large ϵ : less iterations with the cost of more approximate solution
- How to choose ϵ in practice?
 - Try various values to achieve balance between cost and precision
 - Again use some type of **cross-validation** framework
- **Hyperparameters**: Parameters set before the beginning of the learning process (e.g. α , ϵ in gradient descent)
- **Hyperparameter tuning**: The process of choosing a set of optimal hyperparameters for the learning process
- **Model parameters**: The parameters learned during the learning process (e.g. weights $\mathbf{w}$ in linear regression)